

Módulos 08 e 09 — Respostas Conceituais

Trilha de Docker | UniSENAI 2026

Módulo 08 — Exercício 4: Cache de Camadas do Docker

Cada instrução no Dockerfile (**FROM**, **COPY**, **RUN**, etc.) gera uma **camada imutável** na imagem. O Docker armazena essas camadas em cache localmente. Quando você reconstrói a imagem, o Docker verifica camada por camada: se a instrução e os arquivos envolvidos **não mudaram**, ele reutiliza a camada do cache em vez de reexecutar o comando.

Exemplo prático do impacto:

- FROM node:18-alpine # camada 1 → cacheada (raramente muda)
- WORKDIR /app # camada 2 → cacheada
- COPY package*.json ./ # camada 3 → só invalida se package.json mudar
- RUN npm install # camada 4 → reutiliza cache se camada 3 não mudou
- COPY . . # camada 5 → invalida sempre que o código muda
- CMD ["node", "server.js"] # camada 6 → cacheada

Numa mudança típica de código (só o **server.js** mudou), o Docker reutiliza as camadas 1 a 4 e só reexecuta a partir da 5. O **npm install** — que pode levar minutos — é ignorado. Builds que levariam 3 minutos passam a levar 5 segundos.

Regra de ouro: coloque as instruções que mudam com menos frequência **no topo** do Dockerfile e as que mudam mais frequentemente **no final**. Qualquer mudança invalida o cache de todas as camadas abaixo dela.

Módulo 09 — Exercício 3: Importância do Versionamento de Imagens

O versionamento de imagens Docker funciona pelo sistema de **tags** (**imagem:versão**). Sem ele, todos usariam sempre a tag **latest**, o que causa uma série de problemas em produção.

Problemas sem versionamento:

- Um deploy acidental de uma versão com bug afeta todos os ambientes imediatamente
- É impossível saber qual versão exata está rodando em cada servidor
- Rollback se torna difícil: não há como voltar para "a versão de ontem"

Benefícios do versionamento semântico (**1.0.0**, **1.1.0**, **2.0.0**):

Situação	Com versão	Sem versão (latest)
Bug em produção	<code>docker pull minha-app:1.0.9</code> (rollback imediato)	Impossível voltar com segurança
Múltiplos ambientes	dev usa 2.1.0-beta , prod usa 2.0.3	Todos usam o mesmo latest
Rastreabilidade	Log mostra exatamente qual versão causou o incidente	Impossível determinar
Equipe grande	Cada PR gera uma tag (1.2.0-pr-42)	Conflitos de latest

Estratégia recomendada:

- `seuusuario/app:latest` → sempre aponta para a versão mais recente estável
- `seuusuario/app:1.2.3` → versão exata (imutável, nunca sobrescrever)
- `seuusuario/app:1.2` → aponta para o patch mais recente da minor 1.2
- `seuusuario/app:dev` → versão de desenvolvimento (instável)

-